

Architecture and Stream-Processing Load

How London Live is built, what flows through each stage, and what each stage costs to run

Hongwei Peng

1 October 2026

Abstract

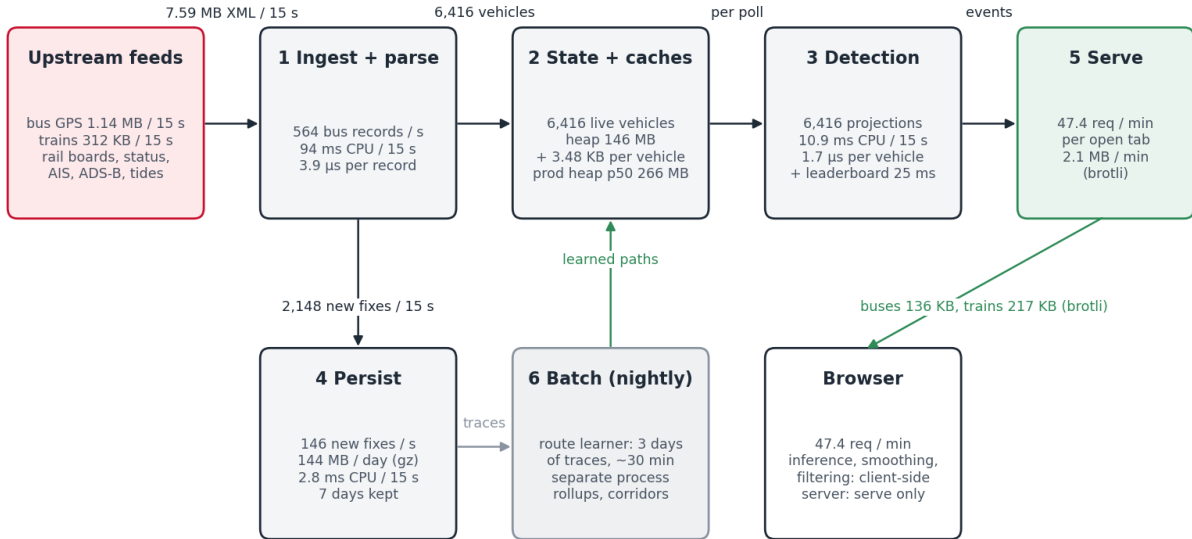
London Live (london.pengrubin.com) turns a dozen public transport feeds into one live map of London. This document describes it as a stream-processing system: the six stages every record passes through, how much data flows through each stage per second and per day, and what each stage costs in CPU, memory and storage. The body is written in terms of named parameters (records per poll, live vehicles, viewers, retention) so that it stays valid as the code and the city change; the measured values that calibrate the formulas are collected in dated reference tables and figures. Calibrated on the London deployment on 2026-09-30 (daytime) and 2026-09-29 (night), with the Dubai deployment as a second, near-empty point.

Table of contents

Summary	2
1 Scope and how to read this document	3
2 Architecture overview	3
2.1 One process, six stages	3
2.2 The rules that shape the load	4
2.3 Where the computation is	4
2.4 Deployment shape	5
3 Input streams	5
3.1 Two kinds of trigger	6
3.2 The proxy path	6
3.3 Where the bytes are	6
4 Stage by stage	9
4.1 Ingestion and parsing	9
4.2 In-memory state and caches	10
4.3 Online detection	11
4.4 Persistence	12
4.5 Serving and egress	12
4.6 Batch jobs	13
5 Client-side processing	14
5.1 What a tab requests	14
5.2 What a tab computes	14
6 End-to-end budget	15
6.1 Latency from source to screen	16
7 Scaling	16
7.1 Ten times the audience	16
7.2 A national feed	17
7.3 A year of traces on the server	17
8 Limits of the model	17

References	17
Appendix A. Reference deployment measurements	18
Appendix B. Stages to code	20

Summary



Measured 2026-09-30 12:40 to 13:13 UTC (weekday daytime): production feeds, local replica on Apple M4, Node 24.13.

Whole process over a weekday: 476 ms CPU per 15 s poll on average (3.17% of one core), 286 to 596 ms by hour. Production heap p50 266 MB, RSS p50 858 MB.

Figure 1: The pipeline with its measured rates. Arrows carry data per poll; boxes carry the cost of one stage. Server CPU is a function of the feeds, not of the audience.

- **One process, six stages.** Ingest and parse; in-memory state and caches; online detection; persistence; serving; nightly batch. Everything except the nightly route learner runs in a single Node.js process.
- **The input is large, the output is small.** The bus feed alone delivers 43.7 GB of XML per day, 564 records per second; after de-duplication about 101 new GPS fixes per second are kept on average over a day (146 at midday), 144 MB per day compressed on disk.
- **The whole live path costs about 3.17% of one CPU core** at London scale, averaged over a weekday on the reference machine, a laptop: 476 ms per 15 s poll, between 286 ms in the small hours and 596 ms in the afternoon. About 269 ms of that is fixed by the feed, which carries almost the same bytes at 01:00 as at 13:00; the rest follows the vehicles in service and the size of the day's leaderboard.
- **Memory is a linear function of two counts.** Heap $\approx$ 146 MB + 3.48 KB per tracked vehicle + 0.326 KB per leaderboard entry (R^2 0.79 over 2,595 five-minute samples); production runs at a median of 266 MB heap and 858 MB resident.
- **Viewers add serving work only.** Position inference, smoothing and filtering run in the browser; every tab receives the same cached bodies, so no viewer adds ingestion, detection or upstream calls. A tab costs the origin 47.4 requests and about 2,064 KB per minute after compression, 63% of it the train-prediction body, and about 90 ms of main-thread time per 15 s for serialisation, compression and the garbage they make.
- **What would break first** is not CPU. It is memory ceilings on the single process, upstream rate limits, and stale bodies served without an age limit; each has been hit once, and each is now bounded where it bit.

1 Scope and how to read this document

This is the second of three technical notes on London Live. The first describes how the bus GPS feed becomes route shapes, on-map positions and diversion events; the third will describe how train positions are inferred from arrival countdowns. This one describes the system that runs them: what components exist, what flows between them, and what each costs.

The body is durable by construction. It names components by role, not by file; it states costs as formulas in a small set of parameters (Table 1); and it gives no dates. Measured values appear in the body only as the calibration of those parameters, and every one of them is traceable to a dated measurement in [Appendix A](#) or a figure caption. When the code or the city changes, the reference section is re-measured, the numbers are regenerated, and the body stands.

Readers who want the numbers should read the Summary and Section 6. Readers who want to size a deployment of their own should read Section 3 and Section 4, substitute their own parameters, and check the limits in Section 8. Readers who want to find the code should use [Appendix B](#).

Table 1: Parameters used throughout. Values are the reference deployment’s; [Appendix A](#) gives their measurement.

Symbol	Meaning	London value (daytime)
R_{bus}	records carried by one bus-feed response	8,454
V	live vehicles after the parser’s freshness cut	6,416 (day), 3,314 (night)
T_{bus}	bus poll interval	15 s
P	train predictions per fetch	8,084
N_{routes}	learned route-directions	1,953
F	new GPS fixes kept per day (daily mean 101 per second; midday 146)	8,768,876
S	vehicle states held by the detector (30-minute window)	7,402 median, 10,125 p95
L, L_{open}	leaderboard entries: closed per day, and held across the three open periods	22,930 per day; 195,381 to 475,489 open
U	open browser tabs	audience-dependent
D	trace retention on the server	7 days, capped at 2 GB ($\approx$ 2.8 days)

2 Architecture overview

2.1 One process, six stages

London Live is a stream-processing system in the sense of Stonebraker et al. (2005): records arrive continuously, every record is handled once as it passes, the state that matters is kept in memory, and the answer to any query is whatever the state says at that instant. It is also, deliberately, a small one. One Node.js process does everything on the live path; a second process is spawned once a day for the batch stage and exits when it is done. There is no queue, no database and no worker pool between the feeds and the browser. The reason is the shape of the load: at London scale the whole live path fits in a few percent of one CPU core (Section 6), so any component added for scale would cost more to run than the work it distributed.

Figure 1 shows the six stages. Every record from every feed passes through the first two; the bus feed alone passes through all of them.

1. **Ingestion and parsing** (Section 4.1). Timers fetch each upstream on its own cadence; push feeds are read as they arrive. Bodies are decoded and reduced to typed records, and the raw body is released.
2. **In-memory state and caches** (Section 4.2). The current picture of the network: one table per vehicle kind, one state per vehicle the detector is following, one index per learned route, one bucket per leaderboard period, and a small cache per upstream response.

3. **Online detection** (Section 4.3). Work done on each new record while it is in hand: projecting a bus onto its learned route and testing whether it has left it, ranking vehicles by distance, placing a rail service on the network graph, resolving a disruption’s text to a place.
4. **Persistence** (Section 4.4). The append-only record of what the feeds said, and the small derived files the next day’s batch and the map need.
5. **Serving and egress** (Section 4.5). HTTP responses to browsers, built from stage 2’s state, compressed per request.
6. **Batch** (Section 4.6). Nightly and hourly jobs that read stage 4 and write the learned geometry and summaries that stages 2 and 3 load at the next boot or rebuild.

The stages are not modules with interfaces between them. They are a way of reading the one process so that each piece of work can be given a cost and a scaling law, which is what this document is for. [Appendix B](#) maps them back to files.

2.2 The rules that shape the load

Nine design rules recur in the code’s comments. Six of them decide what the system costs, and they are the ones a reader sizing a similar deployment should copy.

- **Every viewer-triggered upstream has its own budget.** Calls to each provider are counted against a per-minute ceiling, and the ceilings are separate, so a burst of interest in one layer can never starve another. The budgets are hard: when one is spent, the response is the last good body, and if there is none, an error. Timer-driven streams (the bus feed, the vessel stream, the status recorders) are bounded by their cadence instead. Upstream traffic therefore has a known maximum however many people are watching.
- **Piggyback, never double-spend.** Anything the server needs for itself that a browser also asks for is fetched through the same cache with the same key. The leaderboard’s train sampling uses the same prediction body the map does; the rail sampler uses the same departure boards. The samplers add no upstream calls while a viewer is present and set the floor when none is.
- **Bound memory by entries, not by hope.** Every cache and every table has a maximum entry count or a time-to-live, and the large bodies (train predictions) have a small entry cap because one entry is several megabytes. The one place this was not done, retained substrings pinning whole feed bodies, is the one place memory was lost.
- **Serve stale rather than nothing, but bound staleness where meaning decays.** A vehicle position from ten minutes ago is worse than no position; a tide reading from ten minutes ago is fine. The routes whose meaning decays fastest (aircraft, disruptions, stop closures) carry a ceiling on how old a stale body may be; the others serve the last good body at any age.
- **Never throw from a poll loop or a writer.** A failed poll logs and waits for the next tick; a full disk drops the batch. “Losing a batch is fine; blocking or ballooning memory is not.”
- **Pure CPU work must yield.** Long builds over the whole route set are chunked and yield to the event loop so that a rebuild never delays a poll.

The other three, secrets never leave the server, optional features are key-gated rather than absent, and geometry fails closed, shape correctness rather than cost and are described in the code.

2.3 Where the computation is

The most consequential decision is not in the server at all. Vehicle position inference for trains, per-frame interpolation, snapping buses to routes and smoothing them, the classification and dead-reckoning of aircraft, all of it runs in the browser (Section 5). The server’s outputs are the feeds’ records, de-duplicated and re-encoded compactly, with no parameter that depends on who is asking. Two consequences follow. Every open tab receives byte-identical bodies, so an edge cache in front of the origin can absorb repeated requests, and the origin’s own work is the same for one viewer as for a hundred. And the server’s cost is a function of the feeds alone: how many records they carry and how often they are polled.

The single exception is the leaderboard. Ranking vehicles by distance travelled needs a position history that is continuous and the same for everyone, which a browser cannot provide, so one copy of the train inference runs server-side, every 15 s, on the cached body (Section 4.3).

2.4 Deployment shape

The reference deployment is one small cloud instance with a persistent volume, behind a content-delivery edge, with the base-map tiles served from object storage rather than from the process. The process serves the built front-end as static files and the baked geometry (route shapes, station lists, the rail graph) from the same origin, so the browser sees one host. Runtime-written state (traces, learned routes, leaderboard days) goes to the volume through a resolver that falls back to the working directory, and if that fails too, logs and drops the write rather than throw, because a bad volume must not stop a poll. [Appendix A](#) gives the instance and provider used for the measurements.

3 Input streams

Table 2 lists every stream the London deployment reads, with its transport, cadence, trigger and measured volume. Two of them carry almost all the bytes.

Table 2: Input streams of the London deployment. “Viewer misses” means one fetch per cache expiry however many viewers are present. Bus and prediction volumes measured on 2026-09-30, the rail board, status and tide rows on 2026-09-29; the vessel stream’s byte rate is not visible to the process’s counters.

Stream	Provider and form	Cadence	Trigger	Per poll	Per day (decoded)
Bus positions	BODS SIRI-VM, XML over HTTPS (gzip) (CEN, 2015 ; Department for Transport, 2026)	15 s	timer	8,454 records, 1.14 MB wire, 7.59 MB decoded	43.7 GB
Train arrival predictions	TfL Unified API, JSON (Transport for London, 2026a)	15 s floor, 8 s cache	sampler timer + viewer misses	8,084 predictions, 312 KB wire, 7.4 MB decoded	42.7 GB at the floor
National Rail departure boards	Darwin LDBWS, JSON (Rail Delivery Group, 2026)	one of 17 hubs per 15 s; 45 s cache	sampler timer + viewer misses	102 KB upstream, 14 to 20 KB served	0.59 GB
Line status (window form)	TfL Unified API, JSON	120 s	recorder timer	21 KB wire, 641 KB decoded	0.46 GB
Road disruptions	TfL Unified API, JSON (Transport for London, 2026b)	6 h recorder; 120 s cache	timer + viewer misses	34 KB wire, 271 KB decoded, 117 items	0.2 GB
Bus stop closures	TfL Unified API, JSON, plus stop-point batches	600 s cache	viewer misses	421 stops	0.04 GB
Vessels	AIS over WebSocket	push	connection	109 vessels in the table	not measured
Aircraft	ADS-B aggregator, JSON	10 s cache	viewer misses	104 aircraft	small
Tide gauges	Environment Agency, JSON	300 s cache	viewer misses	10 gauges	small
Popup detail	TfL stop, vehicle, crowding, lifts, bike points	8 to 600 s caches	viewer clicks	one object	audience- dependent

3.1 Two kinds of trigger

Streams are either **timer-driven** or **viewer-triggered**, and the distinction is what makes the load predictable.

A timer-driven stream is fetched on a fixed cadence whether or not anyone is watching: bus positions every 15 s, line status every 120 s, the leaderboard’s prediction and board samples every 15 s, the road-disruption recorder every six hours. Their cost is a constant of the deployment. They exist because the system’s derived products, traces, diversion events, leaderboard distances, status history, must be continuous.

A viewer-triggered stream is fetched only when a browser asks and the cached copy has expired. Its cost has a floor of zero and a ceiling of one fetch per cache lifetime, whatever the audience. Between one viewer and many the upstream call count does not change; only the origin’s serving work does.

The train-prediction stream is both. The sampler sets a floor of one fetch per 15 s; a viewer polling every 10 s against an 8 s cache raises it to one per 8.8 s. On the generic proxy routes more viewers cannot raise the rate further, because a miss while a fetch is in flight joins that fetch rather than starting another. The prediction route is hand-written and lacks that guard, so several first-time viewers arriving within one 8 s window can each trigger a fetch; the budget still caps the total, and the gap is listed in Section 8.

3.2 The proxy path

Most viewer-triggered upstreams go through one generic route with a fixed decision order, and the order is the load model for that class (the all-lines prediction route and the rail board route are hand-written and follow the same shape without the in-flight join):

1. Validate the query. Bad input never reaches an upstream.
2. Fresh cache hit: serve it. The common case; costs serialisation and compression only.
3. Miss with a fetch already in flight for the same key: wait for it and serve its result. One upstream call serves every concurrent miss.
4. Miss, no fetch in flight: if the key failed within the last 30 s, fall to step 6 without touching the budget; otherwise try to consume one unit of that upstream’s per-minute budget, and if the budget is spent, fall to step 6.
5. Fetch. On success, cache and serve. On a thrown error, arm a 30 s per-key back-off and fall to step 6.
6. Serve the stale entry if one exists and is within that route’s staleness ceiling; otherwise return 429 (budget) or 502 (upstream).

Upstream error bodies are stripped of credentials before anything is logged or forwarded, and a non-200 upstream response is passed through with the key redacted rather than cached. Each response carries a header saying which branch served it, which is how the shares of hits, misses and stale bodies in [Appendix A](#) were measured.

3.3 Where the bytes are

By decoded volume the bus feed and the prediction feed are of a size: 43.7 GB and 42.7 GB per day. On the wire they are not: the bus feed’s XML compresses from 7.59 MB to 1.14 MB, the predictions’ JSON from 7.4 MB to 312 KB, so inbound traffic is 6.5 GB per day of buses against 1.8 GB of predictions. Everything else together is under a gigabyte a day. Inbound bytes are not what costs money in this deployment (Section 6), but decoded bytes are what the parser and the garbage collector see, and both feeds are large by that measure.

The bus feed is large for a reason worth stating. A SIRI-VM datafeed response contains every vehicle activity the operators’ systems currently hold for the bounding box, and operators keep reporting a vehicle for some minutes after it stops moving. By day 6,416 of 8,454 records are fresher than five minutes; late in the evening 3,314 of 8,273 are (Figure 2), and at 01:00 about 611 are while the feed is still 1,082 KB on the wire against 1,144 KB at the morning peak (Figure 3). The feed’s size is set by how many vehicles have reported recently, not by how many are in service. What the system then does with the records does follow the vehicles in service: over a full weekday the process’s CPU runs between 286 and 596 ms per poll while the feed’s bytes move by 1.1:1, so the live load is a fixed part set by the feed plus a part set by V (Section 4.1, Section 6).

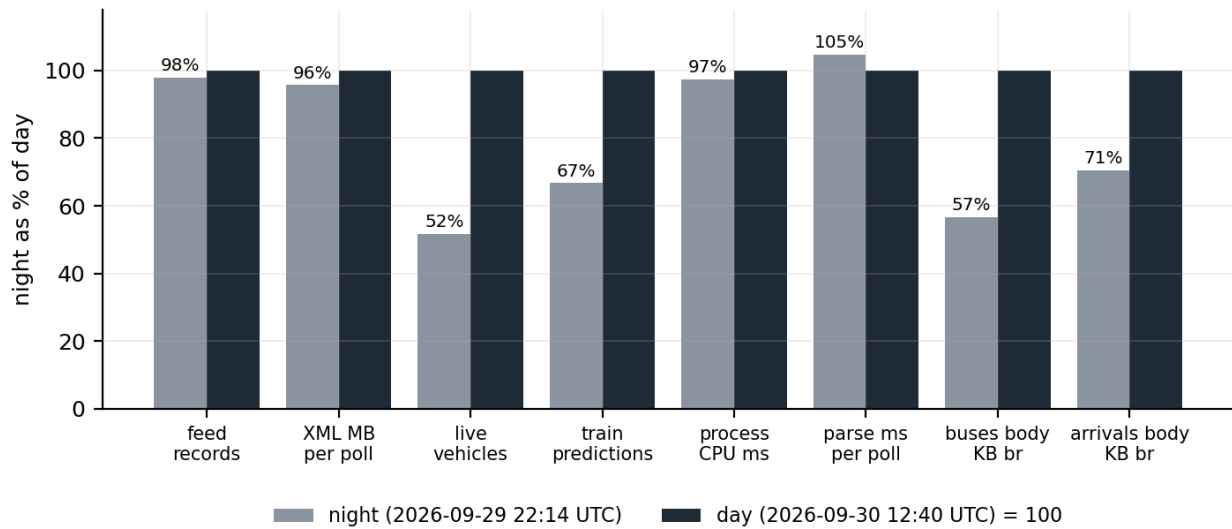


Figure 2: Night as a share of day for the quantities that drive load, from the two half-hour calibration windows. The feed's records and bytes barely move while live vehicles and served bodies halve; the two CPU figures happen to match because the evening window still had half the day's vehicles. Measured 2026-09-29 and 2026-09-30.

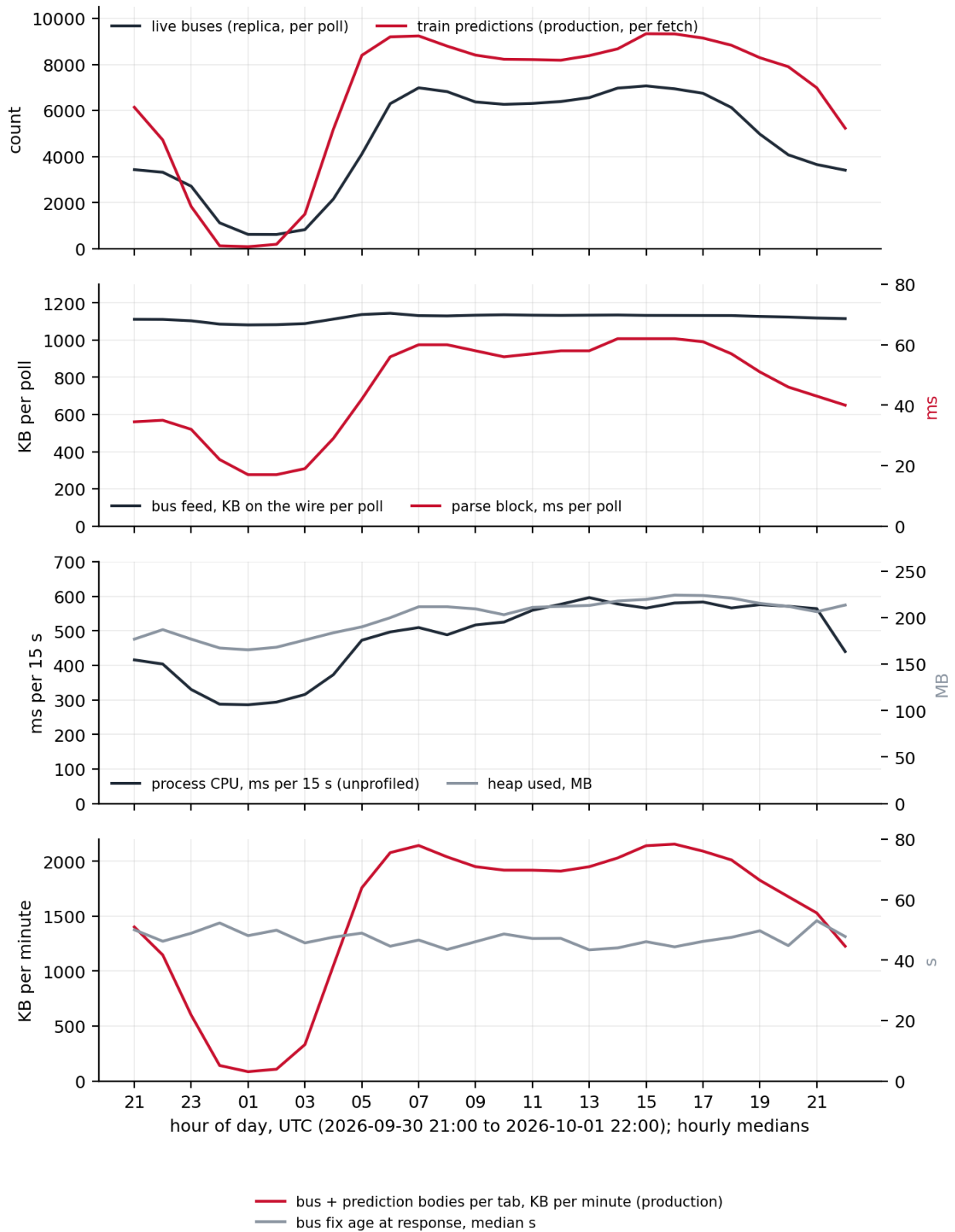


Figure 3: One weekday of the reference deployment, hourly medians. Top: live buses on the replica and train predictions on the production origin. Second: the bus feed's bytes per poll are flat while the parse block follows the live count. Third: process CPU and heap on the replica. Bottom: what one tab downloads per minute and the age of the bus fixes it receives. Replica run 2026-09-30 21:00 UTC to 2026-10-01 22:00 UTC, 5,907 polls, with production sampled every 10 minutes.

4 Stage by stage

Each stage is described on the same card: its function, its input and output rates, its CPU per bus poll (the 15 s period everything else is measured against), its memory and storage, and the parameter it scales with. CPU figures are main-thread milliseconds from a sampling profiler on a local replica fed by the production upstreams; the profiler roughly doubles process CPU, so proportions come from the profile and absolute totals from an unprofiled control run ([Appendix A](#)). Figure 4 gives every stage’s share at once.

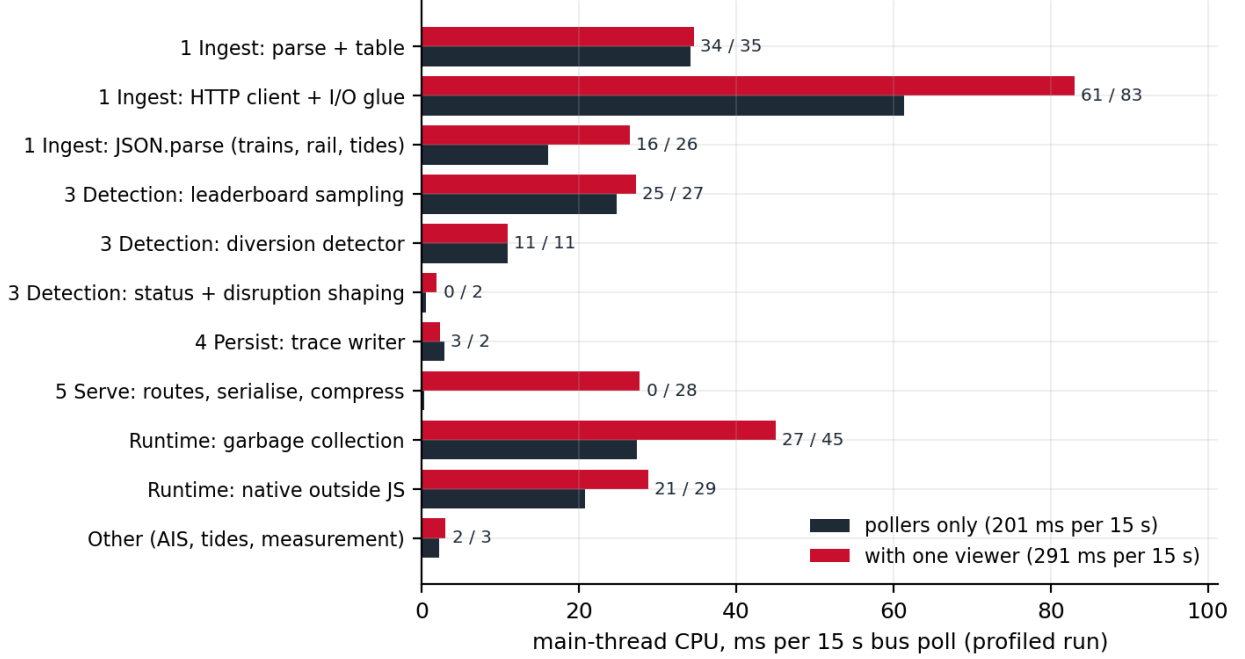


Figure 4: Main-thread CPU per 15 s bus poll by stage, from the profiled local run, without and with one synthetic viewer polling at the front-end’s intervals. Ingestion is more than half; serving appears only when a viewer is present. Measured 2026-09-30.

4.1 Ingestion and parsing

Function. Fetch each upstream on its own timer, decode the body, and turn it into typed records. The bus feed is the only one large enough to matter: one SIRI-VM XML document per poll containing every vehicle the operator systems reported in the last few minutes, whether or not it moved. The parser walks the document as a string, cutting out the eleven fields the system uses and copying each one into fresh memory before the document is released. That copy is the single most important line in the stage: the JavaScript engine’s substring returns a view onto the parent string, and a retained view keeps the whole multi-megabyte document alive. An earlier build of this system lost its memory to exactly that.

Input rate. R_{bus}/T_{bus} records per second and one document of B_{bus} bytes per poll. For London, R_{bus} is about 8,454 and B_{bus} is 7.59 MB decoded (1.14 MB on the wire), or 564 records and 43.7 GB per day. The train-prediction feed is the second largest: 7.4 MB of JSON per fetch, 8,084 predictions, fetched every 15 s by the leaderboard sampler and more often when viewers are watching.

Output rate. V live vehicles per poll, after dropping records older than five minutes: 6,416 by day, 3,314 in the evening, 611 at 02:00 UTC. The document size does not follow that collapse: at 01:00 the feed still carries 1,082 KB and nearly the same 8,273 records, most of them stale. This is the stage’s central property. **Its input cost is set by what the feed carries, R_{bus} ; only the records it keeps, V , cost anything downstream.**

CPU. Per bus poll: parse and table build 32.7 ms, that is 3.9 μ s per record; TLS, sockets and inflate 39.6 ms and the HTTP client 21.8 ms, both shared across all upstreams but dominated by this feed’s bytes; JSON parsing of the upstream bodies, of which the prediction body is nearly all, 16.1 ms. The parse block the process logs, which spans the scan, the table build and the hand-off of each kept record to the trace writer and the detector, splits the two costs cleanly over a day: 14.4 ms fixed plus 6.9 μ s per live vehicle (R^2 0.78

over 5,907 polls), 52 ms at the median and 68 ms at the 95th percentile. The scan itself is the fixed part; the per-vehicle part is the work the records go on to.

$$\text{CPU}_{\text{ingest}} \approx c_{\text{parse}} \cdot R_{\text{bus}} + c_{\text{bytes}} \cdot B_{\text{bus}} + c_{\text{json}} \cdot B_{\text{pred}} + c_{\text{keep}} \cdot V \quad (1)$$

with $c_{\text{parse}} \approx 3.9 \mu\text{s}$ per record, $c_{\text{keep}} \approx 6.9 \mu\text{s}$ per kept vehicle and $c_{\text{json}} \approx 2.2 \text{ ms}$ per MB on the reference machine.

Memory. One document in flight, B_{bus} , plus the typed records for one poll. Nothing from this stage survives the poll except what the next stage copies.

Storage. None.

Scales with: R_{bus} and B_{bus} for the scan, that is with the feed’s coverage and how long operators keep reporting parked vehicles; V for the hand-off; not with viewers. Doubling the poll rate doubles the whole stage.

4.2 In-memory state and caches

Function. Hold the current picture of the network, and nothing older than it needs. Four kinds of state matter for size. The **vehicle tables**, one per feed, replaced wholesale on each poll, plus a pre-built compact array of the bus table in wire form so that serving does not rebuild it. The **detector states**, one per bus seen in the last 30 minutes, holding the last few positions along its route and any excursion in progress; this is the set S , larger than V because it remembers vehicles that have gone quiet. The **route indexes**, one per learned route-direction, N_{routes} of them, each a spatial grid over the route’s vertices so that projecting a fix costs a handful of cell lookups; and, beside each, a 256-sample shape gate. The **leaderboard buckets**, three open periods (day, week, month), each holding one running total per distinct vehicle seen in the period, L per day. Around these sit the response caches: one entry per upstream key, bounded by entry count, with the multi-megabyte prediction body capped at four entries.

Input rate. Every record stage 1 keeps, V bus records per poll and P predictions per fetch, is written into this stage; the tables turn over completely each poll.

Output rate. Reads by stages 3 and 5: one pass over V per poll for the detector and the leaderboard, one serialisation of a table per served request.

CPU. Table construction is inside the parse figure of Section 4.1. The state’s own cost is indirect and shows up as garbage collection: 27.4 ms per poll with no viewer, 45 ms with one, because each poll allocates a fresh table and each served response allocates a body.

Memory. This is the stage the heap belongs to, and it is where the durable formula lives. Over 2,595 five-minute samples of the production process (15.5 days of uptime), heap used is fitted by two counters that the process reports about itself (Figure 5):

$$\text{heap} \approx 146 \text{ MB} + 3.48 \text{ KB} \cdot S + 0.326 \text{ KB} \cdot L_{\text{open}} \quad (2)$$

with $R^2 = 0.79$ and a residual standard deviation of 13.47 MB. The intercept is the baked geometry, the route indexes and the runtime; the first slope is everything held per followed bus, tables, detector state and trace bookkeeping together; the second is the leaderboard’s per-vehicle totals across the three open periods, L_{open} , which grows through a month and drops when the month closes. The counters themselves ran from 56 to 10,288 vehicle states and 195,381 to 475,489 leaderboard entries in the window, so the fit is interpolating, not extrapolating. Production sits at a median of 266 MB heap (320 MB at the 95th percentile); the day-to-night swing in Figure 5 is the S term.

Resident memory is larger, 858 MB at the median, and does not fit the counters. It includes 73 MB of buffers outside the JavaScript heap (bodies in flight, compression state) and whatever the runtime has reserved and not returned. The gap between heap and resident size is what a memory ceiling must be set against.

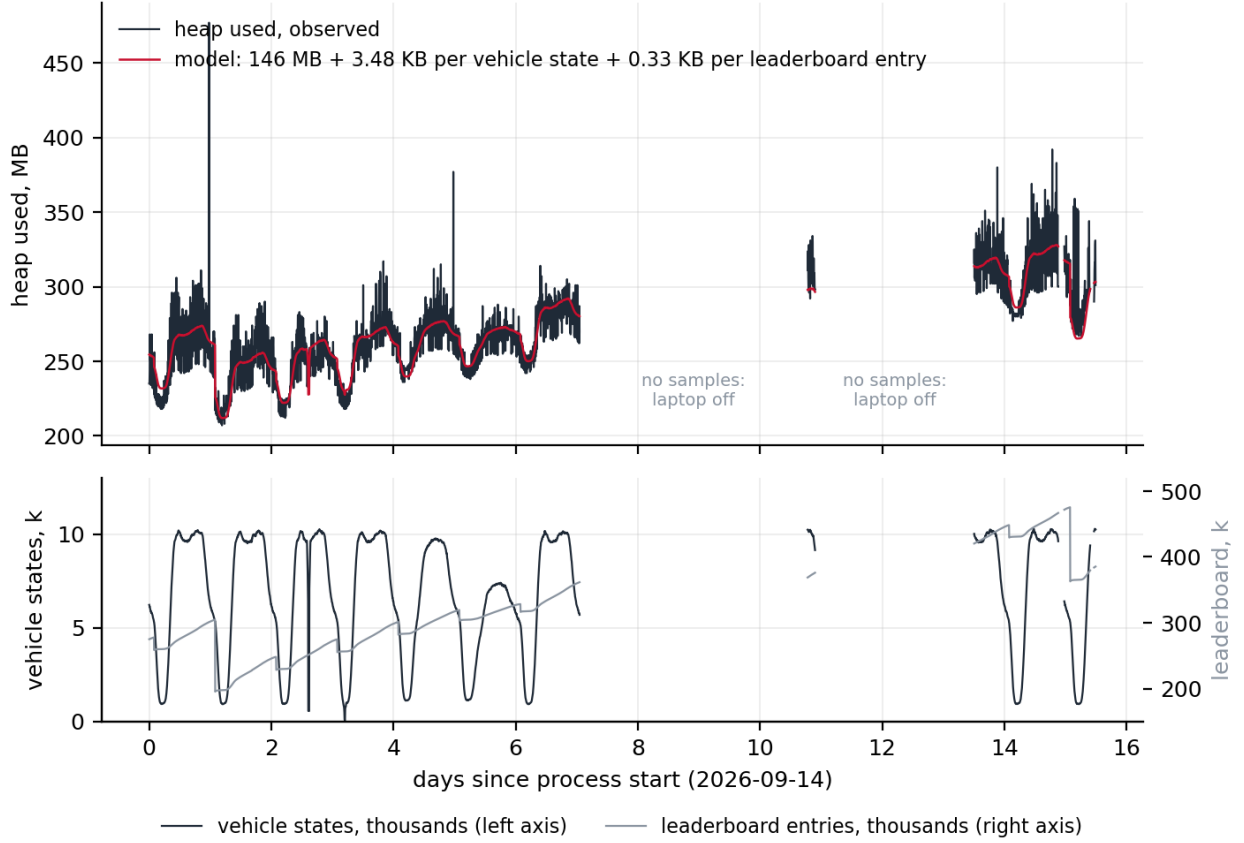


Figure 5: Production heap over 15.5 days of one process, observed and as predicted by Equation 2 from the two counters below it. The daily wave is vehicle states; the slow rise is the open leaderboard periods. Sampled every five minutes from the process’s health endpoint; gaps are where the sampler was offline.

Storage. None; this stage is the reason stage 4 exists.

Scales with: S (about $1.28 V$ for a 30-minute window, measured), N_{routes} for the intercept, and L_{open} for the slow term. Not with viewers, and not with R_{bus} : a stale record is dropped before it reaches this stage.

4.3 Online detection

Function. Do, once per record, the work that produces something the feed did not carry. Four detectors run here. The **diversion detector** projects each routed bus onto its learned route through the grid index, updates its state, and applies the excursion rules that open, extend, confirm and retire events (the rules are the subject of the first document in this series). The **leaderboard sampler** advances every vehicle’s distance with a haversine step per fix, and for trains, which have no fixes, runs the same position inference the browser runs, once, on the shared prediction body. The **rail sampler** merges one departure board per tick into per-service timelines and places each service on the baked rail graph by clock, with a shortest-path search on a cache miss. The **status and closure shapers** turn free-text disruption notices and stop-closure lists into map geometry, once per cache miss rather than once per poll.

Input rate. V bus records per poll into the first two; P predictions per fetch into the inference; one board per tick into the rail sampler; one notice list per cache expiry into the shapers.

Output rate. Diversion transitions: 5,411 state changes per day, a few dozen live events at any time. Leaderboard: 22,930 vehicle totals per day. Rail: one position per tracked service per tick.

CPU. Per poll: diversion detector 10.9 ms, that is 1.7 μ s per live bus; leaderboard sampling 24.8 ms early in a day, of which train inference is about 5 ms and the once-a-minute synchronous write of all open buckets is most of the rest; shaping under 1 ms. The write grows with the day: it serialises every open bucket, so its cost is proportional to L_{open} , and over the 24-hour run the process’s CPU rose by about 6.7 ms per poll per hour of uptime at constant V , with the live leaderboard file at 7 MB by the end of the day. The detector’s

per-bus cost is small because the grid makes projection constant-time and the shape gate, a 256-sample median updated on every routed fix, is consulted only when an excursion is being judged.

$$\text{CPU}_{detect} \approx c_{proj} \cdot V \cdot (1 + N_{ev}) + c_{infer} \cdot P + c_{persist} \cdot L_{open}/4, \quad c_{proj} \approx 1.7 \mu\text{s} \quad (3)$$

where N_{ev} is the number of open events each fix is tested against, and the last term is the leaderboard persist amortised over the four polls in its minute.

Memory. Inside the S term of Equation 2: per vehicle at most 32 recent positions along the route, at most 2,000 excursion fixes while an excursion is open, and a 30-minute time-to-live. Per event at most 500 members and 12 route segments. Events are rare enough that they add nothing measurable to the heap fit.

Storage. Diversion transitions 1.31 MB per day; leaderboard 2.3 MB per closed day (Section 4.4).

Scales with: V and N_{ev} for the detector, P for the inference, L_{open} for the persist. Not with viewers.

4.4 Persistence

Function. Write down what the feeds said, once, in the cheapest form that the batch stage can read back. The **trace writer** is the one that matters. On each poll it compares every live bus’s report time with the last one it wrote for that vehicle and appends only the fixes whose time advanced: a bus that reported once and is still being carried by the feed costs nothing after the first line. Fixes are buffered and flushed every 5 s as JSON lines into one file per day. Smaller writers append diversion transitions, line-status and road-disruption snapshots (only when the payload changed or a heartbeat is due), and rewrite the live leaderboard file each minute. Every JSON file is written to a temporary name and renamed, so a crash leaves the previous file intact.

Input rate. V live records per poll.

Output rate. $F = 8,768,876$ new fixes per day, a daily mean of 101 per second or 1,522 per poll. At midday the rate is higher, 2,148 per poll or 146 per second, 0.335 per live bus per poll: a London bus reports about once every 45 s and the feed is polled three times as often. The de-duplication is what keeps the disk rate about a quarter of the feed’s record rate.

CPU. 2.8 ms per poll, about 1.3 μs per fix written. The comparison is a map lookup per live vehicle; the write is buffered.

Memory. Two maps of last-written time and position per vehicle id, inside the S term; a line buffer capped at 500,000 lines so that a stalled disk cannot grow it without bound.

Storage. 86.6 bytes per fix as written, 763 MB per day; 16.4 bytes gzip-compressed as archived, 144 MB per day. Retention on the server is the smaller of $D = 7$ days and a 2 GB cap on the directory, and at London’s rate the cap binds first: it holds about 2.8 days, so the nightly learner’s three-day window is in practice slightly short of three days. The archive off the server keeps everything. Everything else the process writes is under 10 MB a day put together (Table 7).

$$\text{disk}_{server} \approx \min(b_{fix} \cdot F \cdot D, \text{cap}), \quad b_{fix} \approx 86.6 \text{ B raw, } 16.4 \text{ B compressed} \quad (4)$$

Scales with: F , which is V times the operators’ report rate, and D or the cap. Not with R_{bus} and not with viewers.

4.5 Serving and egress

Function. Answer HTTP requests from stage 2’s state. Three route kinds. **Direct-table routes** serialise a pre-built array (buses, vessels, bikes). **Proxy routes** serve cached upstream bodies, shaped or raw, through the path of Section 3.2. **Derived routes** build a body from detector state on each request (diversions, leaderboard, coverage). Every response is compressed per request, brotli at a low quality setting or gzip by what the browser accepts (Alakuijala and Szabadka, 2016), and there is no cache of compressed responses: each request pays its own compression. Cache-control headers are set only where the body changes slowly (learned routes, coverage, diversions); the edge in front of the origin caches those for hours and, by its default behaviour, the live bodies for a few seconds, which is enough to merge simultaneous requests but not to change the per-tab arithmetic below.

Input rate. U tabs, each polling the endpoints of Section 5 on fixed intervals: 47.4 requests per minute per tab, independent of what the viewer is looking at.

Output rate. Per tab per minute, 2,064 KB compressed, of which the prediction body is 63% and the bus body 26% (Figure 6). Per tab per hour, 123.8 MB; per tab-day, 3 GB. The ratio of raw to compressed is what makes this affordable: 33:1 for predictions (917 bytes raw per prediction, 28 compressed), 6.5:1 for buses (137 bytes raw per bus, 21 compressed, on the short-key wire format).

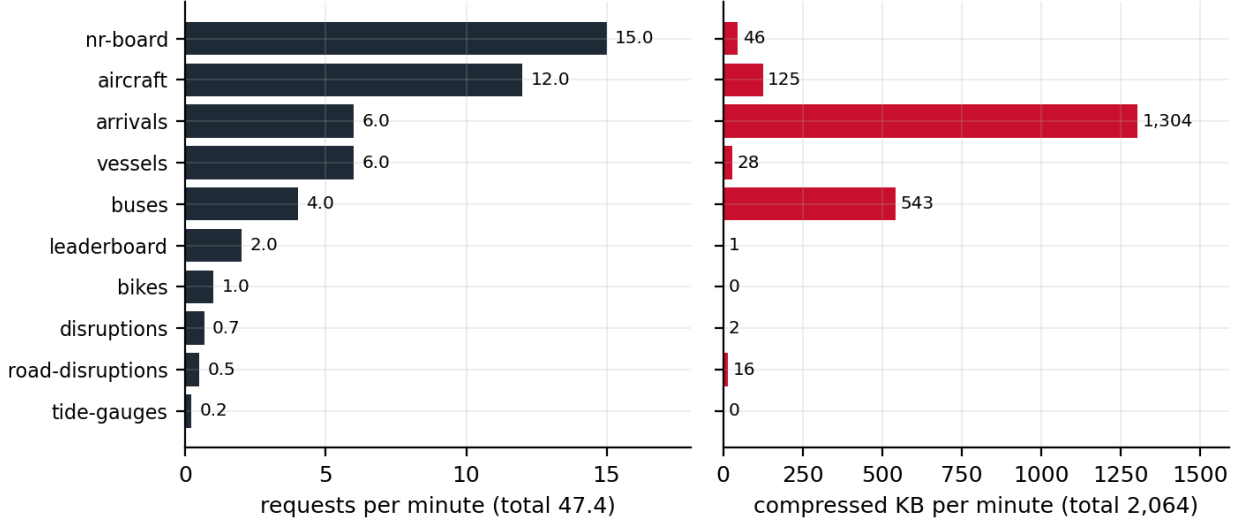


Figure 6: What one open tab costs the origin per minute, by endpoint. Request count is set by the rail board and aircraft layers; bytes are set by the prediction and bus bodies. Body sizes measured on the production origin on 2026-09-30.

CPU. With no viewer, 0.3 ms per poll, the health checks. With one synthetic viewer the main thread gains 90 ms per 15 s: 27.4 ms in the routes themselves, mostly serialisation of the prediction body, and the rest in garbage collection (27.4 to 45 ms), socket and stream glue and the extra prediction fetches the viewer’s misses trigger. Off the main thread the process gains a further 106 ms, which is the compression running on the thread pool. Both figures are from the profiled run and carry the profiler’s overhead, so the unprofiled cost is smaller, about half.

$$\text{CPU}_{\text{serve}} \approx U \sum_i \frac{T_{\text{bus}}}{T_i} (c_{\text{ser}} + c_{\text{br}}) B_i^{\text{raw}}, \quad \text{egress} \approx U \sum_i \frac{B_i^{\text{br}}}{T_i} \quad (5)$$

summing over the polled endpoints i with interval T_i and body sizes B_i . The bus and prediction terms dominate both sums.

Memory. Transient response buffers; the pre-built bus wire array is the one thing kept between requests.

Storage. None.

Scales with: U , linearly, in CPU and egress; and with P and V through the body sizes. This is the only stage that scales with the audience.

4.6 Batch jobs

Function. Turn the trace archive into the geometry and summaries the live path needs, on a schedule that reads freshness stamps rather than a clock, so that a restart never skips a run and never repeats one. The **route learner** is the large job: a separate process spawned once a day that reads three days of traces and fits, for each route-direction key with enough fixes, the polyline that the browser snaps buses to and the detector projects them onto (the method is document 1’s subject). A weekly job fetches the operators’ published route shapes as priors. In-process, once a day, the **coverage builder** resamples every learned route into 25 m pieces and counts overlaps on a 30 m grid, yielding to the event loop every 50 routes; the **detector index rebuild** reloads the learned routes into stage 2’s grids; and hourly, the **rollup writer** streams each completed day of traces once into per-route totals, and trace maintenance enforces retention.

Input rate. Three days of traces per learner run, $3F = 26.3$ million fixes; one day, F , per rollup.

Output rate. In one run, 1,647 keys pass the quality gate out of 2,210 seen in the window; the learned set on disk accumulates across runs to N_{routes} files, 8.3 MB as a compressed snapshot; 2,142 route-direction rollup rows and 303 KB per day; one coverage file.

CPU. The learner takes about 29 minutes of one core on the reference machine, in its own process, so the live path is unaffected except through disk contention. The coverage build is the largest transient the live process itself runs; it is chunked to yield to the event loop, and its duration has not been measured separately.

Memory. The learner reads traces in chunks with a budget of four million fixes, about 100 MB, so its footprint is bounded whatever F is. The coverage build is the live process’s highest memory peak of the day: 88 bytes per 25 m piece over the whole learned network, transient.

Storage. The learned snapshot, 8.3 MB per day if archived; rollups, 303 KB per day.

Scales with: F for time, N_{routes} for output and for the coverage peak. Not with viewers.

5 Client-side processing

The browser is where the map is computed. This section describes it only as far as it bears on load: what it asks the origin for, and what it does instead of asking.

5.1 What a tab requests

Each live layer owns a poll loop on a fixed interval, registered with one lifecycle utility so that a hidden page pauses every loop and a page coming back fires each once. Table 3 is the resulting request profile of one visible tab; it is the U -multiplier of Equation 5.

Table 3: One visible tab’s periodic requests. Diversions and stop closures add about 0.9 per minute when their layers are on. Bodies measured 2026-09-30, daytime.

Endpoint	Interval	Requests per minute	Body, compressed
Rail departure board (17 hubs, round robin)	4 s	15	102 KB raw, 2 to 3 KB compressed
Aircraft	5 s	12	10.4 KB
Train predictions, all lines	10 s	6	217 KB
Vessels	10 s	6	4.7 KB
Buses	15 s	4	136 KB
Leaderboard	30 s	2	0.5 KB
Bikes, disruptions, roadworks, tides	60 to 300 s	2.4 together	31 KB at most
Total		47.4	2,064 KB per minute

Three things about this profile are not obvious from the table. First, request count and byte count have different drivers: the rail board and aircraft layers make 57% of the requests and 8% of the bytes, the prediction body 13% of the requests and 63% of the bytes. Second, the prediction body is served with about twice the fields the browser reads, eleven of some twenty-one per prediction, so the largest term in the egress sum has a factor of roughly two available by trimming. Third, the profile is independent of the viewport and of which layers are visible for most layers; only buses, disruptions, diversions and closures stop polling when hidden in the legend. A hidden *page* pauses everything except the leaderboard and an open camera popup.

Outside the periodic profile, a tab fetches learned route shapes on demand, one file per route-direction of up to 200 KB, cached in the browser up to 600 entries. At low zoom every tracked bus is advanced along its route, which means the whole fleet’s routes are requested once; these files carry a one-hour cache header and are the one variable-volume source a tab has.

5.2 What a tab computes

Per prediction body, the browser infers a position for every train on every line from arrival countdowns and the baked branch geometry, and hands each to an interpolator that advances it per frame along its segment.

Per bus body, it updates a per-vehicle model, snaps each new fix to the learned route with hysteresis, and runs a one-dimensional filter along the route so that the icon moves at a plausible speed between fixes. Per rail board, it places services on the network graph with a shortest-path search on a 431-node graph, cached. Per aircraft body, it dead-reckons from the feed’s own timestamps. The symbol layers’ GeoJSON rebuilds are gated to about 15 Hz, and hot paths are allocation-free by design so that a fleet of several thousand buses does not churn the browser’s garbage collector.

None of this touches the origin. It is the reason U appears in Equation 5 and nowhere else in this document.

6 End-to-end budget

Table 4 collects the stages’ costs at London scale into one place. Read it with Equation 1 to Equation 5 for how each row moves.

Table 4: The budget at London scale. CPU from the profiled local run (proportions) and the unprofiled control run (total); memory from production; disk from the archive.

Stage	CPU per 15 s (main thread)	Memory	Disk per day	Scales with
1 Ingest and parse	112 ms	one body in flight	none	$R_{bus}, B_{bus}, B_{pred}$
2 State and caches	indirect, via GC (below)	146 MB + 3.48 KB $\cdot S + 0.326 \text{ KB}$ $\cdot L_{open}$	none	S, N_{routes}, L_{open}
3 Online detection	36 ms	inside S	1.31 + 2.3 MB	V, N_{ev}, P
4 Persistence	2.8 ms	line buffer	763 MB raw, 144 MB archived	F, D, cap
5 Serve	$\approx 90 \text{ ms}$ per viewer (+ $\approx 106 \text{ ms}$ off-thread), profiled	transient	none	U, P, V
6 Batch	29 min per day, separate process	$\leq 100 \text{ MB}$, separate	303 KB + 8.3 MB	F, N_{routes}
Runtime and other	50 ms (GC 27, native code, vessel stream)			
Whole process, no viewer	201 ms main thread (profiled, midday); all threads 476 ms = 3.17% of one core as a weekday mean, 286 to 596 ms by hour	266 MB heap, 858 MB resident (production medians)	763 MB, 99% traces	

Four readings of the table.

CPU is not the constraint. The whole live path averages 3.17% of one core over a weekday and never exceeded 635 ms per poll at the 95th percentile of 30-second samples. Over the day it fits 269 ms fixed, plus 26.3 μs per live vehicle, plus 6.7 ms per hour of the open leaderboard day (R^2 0.62); the fixed part is the feed, the second is stages 1 to 4 per kept record, the third is the leaderboard persist. The main thread, the only place where a slow operation can delay a poll or a response, is busy 1.34% of the time in the profiled run and about 23% of the process total; the rest is TLS, inflate, compression and garbage-collection helpers on the runtime’s own threads. Headroom is set not by the average but by the largest single block on the main thread, the bus parse, which is why the national scenario in Section 7 reaches a latency concern long before it reaches a core. For viewers, Equation 5 with the profiled per-viewer cost gives on the order of 77 simultaneous tabs per core, and about twice that unprofiled.

Memory is the constraint, and it is understood. The resident size sets the instance class, and it is the sum of a heap given by Equation 2 and a further few hundred megabytes outside it. The two counters in

the formula are reported by the process, so a ceiling can be chosen from them rather than found by crashing. The system’s one outage of memory was a leak of the kind Equation 2 cannot see, retained substrings, and was found with a heap snapshot, not with this model.

Disk is small and linear. Under a gigabyte a day, nearly all traces, and bounded on the server by the 2 GB cap. The archive off the server is where the growth goes, 53 GB per year compressed.

Egress is the only cost that follows the audience. At 123.8 MB per tab-hour, egress is what a change in audience shows up as first, and the prediction body is where two thirds of it is. The two levers not yet pulled are trimming that body to the fields the browser reads, roughly a factor of two, and setting a deliberate edge cache lifetime on the live bodies equal to their poll interval, so that the edge rather than the origin serves every tab but the first in each interval.

6.1 Latency from source to screen

The chain for a bus position has five delays: the operator’s report interval; the feed’s publication lag; the 15 s server poll; the browser’s 15 s poll; and the browser’s own smoothing, which trades some seconds of lag for continuous motion. Figure 7 measures the first three together as the age of each fix at the moment the server’s response leaves: median 32 s by day (46 s at night, when operators report less often), 47% under 30 s, 7% over two minutes, the tail being vehicles the operators’ systems are slow to forward. Add half a browser poll on average and the display is typically 40 s behind the vehicle. The server contributes at most one poll interval of that; the rest is upstream and by design.

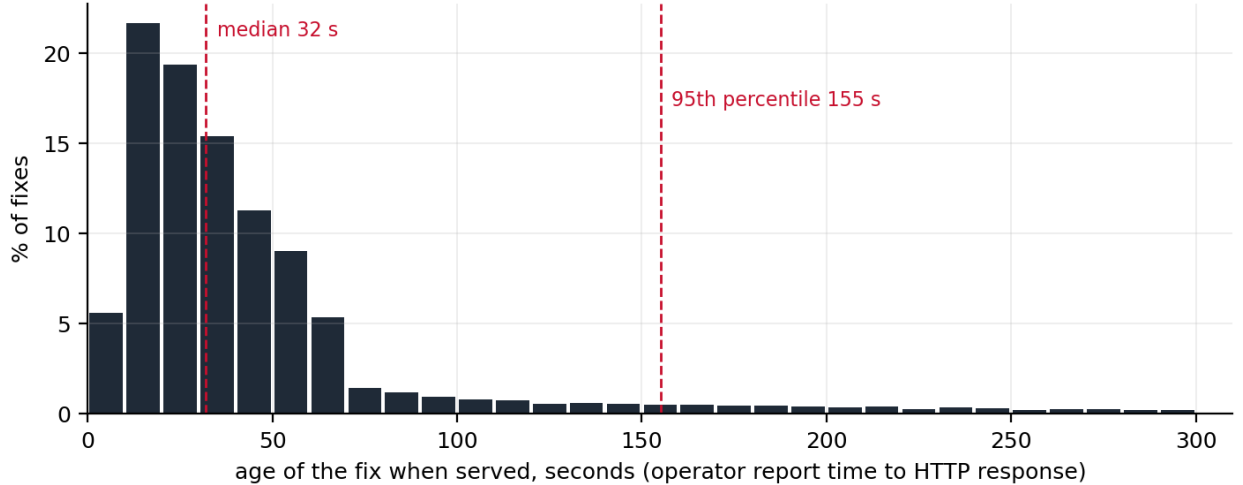


Figure 7: Age of each bus fix when served, over 32,057 fixes in five consecutive daytime responses from the production origin, 2026-09-30. The server adds at most 15 s; the rest is the operators’ report interval and the feed’s forwarding delay.

7 Scaling

The formulas of Section 4 can be evaluated for deployments other than the reference one. Three are worked here, each changing one parameter, each showing which stage moves first. The numbers are the formulas applied to the calibration values, not measurements; they are printed as computed but should be read to about one significant figure.

7.1 Ten times the audience

$U = 10$ simultaneous tabs. Stages 1 to 4 and 6 do not change. Stage 5 adds about 90 ms of main-thread time per tab per 15 s at the profiled cost, taking the main thread from 1.34% to about 7.3% busy, and the whole process to about 19% of a core profiled, roughly half that unprofiled. Egress becomes 29.7 GB per day. Upstream calls change only through the prediction route’s missing in-flight join, and the budget caps even that. What moves first is the egress bill, and the response is edge caching of the live bodies, which turns the U in Equation 5 into the number of edge locations.

7.2 A national feed

The same bus feed without a bounding box carries about 40,000 records per poll, 4.7 times London, so roughly $V \approx 30,357$ live vehicles by day. Stage 1 scales with records: 36 MB of XML per poll, 207 GB per day decoded, and about 445 ms of parse and I/O per poll. Stage 3’s detector goes to about 52 ms. The main thread would be busy about 4% of the time, still comfortable, but with a single 36 MB document decoded and parsed on it every 15 s, the parse block would approach 200 ms of continuous main-thread time, which is where response latency would start to be felt. Memory adds about 109 MB of heap by Equation 2, and the prediction caches are unchanged. Traces grow to 3.6 GB per day as written and 681 MB archived, so the 2 GB cap would hold about 0.6 of a day and would have to be raised for the learner to see its three-day window; the learner itself grows to roughly 137 minutes. The bus body served to browsers grows to 642 KB compressed, which is where the design would need to change: a national map would have to serve by viewport, giving up the identical-body property. What moves first is the parse on the main thread, and the response is a worker thread for stage 1.

7.3 A year of traces on the server

$D = 365$ days, with the cap raised to match. Only stage 4 and the batch stage change. Disk becomes 52.6 GB compressed or 278 GB as written, plus 111 MB of rollups and 478 MB of transitions. Nothing in the live path reads more than a day, and the learner reads three, so CPU and memory do not move. What moves first is the volume size, and the response is what the reference deployment already does: keep D small on the server and archive off it.

8 Limits of the model

The formulas are only as good as their calibration, and the calibration has known edges.

- **CPU was measured on a laptop, not the production instance.** The profiled and control runs used the production feeds but a local machine and a newer runtime (Appendix A). Absolute milliseconds will differ on the deployed hardware; the proportions between stages and the scaling laws are what transfer.
- **One weekday.** The stage-level profile comes from two half-hour windows and the whole-process figures from one continuous weekday (24 hours). The two agree where they overlap, but a weekend and a disrupted day have not been measured, and the 24-hour fit explains 1% of the variance: the rest is garbage-collection timing and the nightly jobs.
- **The heap fit explains 79% of the variance and the samples are autocorrelated.** Consecutive five-minute samples are not independent, so the coefficients’ formal standard errors understate the uncertainty. The residual, 13.47 MB, is the honest error bar. Resident memory has no usable fit; only the heap does.
- **Compression cost is inferred, not measured.** It runs on the thread pool, invisible to the main-thread profiler; the figure quoted is the process-wide difference between the viewer and quiet phases less the main-thread difference, and both contain the profiler’s own overhead.
- **The JSON coefficient is uncertain by a factor of two.** The prediction body was 1.8 times larger by day than by night, but the profiled JSON parse time was the same in both runs, so c_{json} is calibrated from the day run and should be read as an order of magnitude.
- **The audience scenario is a linear extrapolation from one synthetic viewer.** Ten real viewers would also bring popup traffic with viewer-chosen keys and concurrent cache misses, neither of which the synthetic viewer generated.
- **Two streams are unmeasured.** The vessel stream’s message rate is invisible to the process’s request counters, and the aircraft upstream answered the replica with errors in both runs, so its parse and serve cost is absent from the profile.
- **Two fetchers lack the in-flight join.** The all-lines prediction route and the rail board route are hand-written and do not join concurrent misses, so a burst of first-time viewers within one cache window can spend several budget units on identical fetches. It is the one place where the upstream-call ceiling of Section 3.1 is soft; the budget still holds.

References

Alakuijala, J., Szabadka, Z., 2016. Brotli compressed data format (No. RFC 7932). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7932>

CEN, 2015. CEN/TS 15531: Service interface for real-time information (SIRI), part 5: Vehicle monitoring. Department for Transport, 2026. Bus open data service: Developer documentation.
Rail Delivery Group, 2026. [Live departure boards web service \(Darwin\) via the rail data marketplace](#).
Stonebraker, M., Çetintemel, U., Zdonik, S., 2005. The 8 requirements of real-time stream processing, in: ACM SIGMOD Record. pp. 42–47. <https://doi.org/10.1145/1107499.1107504>
Transport for London, 2026b. TfL unified API: Road disruptions.
Transport for London, 2026a. [Unified API: Line arrivals and status](#).

Appendix A. Reference deployment measurements

All figures in the body come from the measurements below. The replica runs used the code at commit 6875e27; the production endpoint and fix-age measurements used the build deployed on the day named. Each is reproducible from the scripts in the `02-architecture/scripts/` directory of the reports repository against the raw files in `data/`.

Deployment. Production: one Railway service (single instance, persistent volume) behind Cloudflare, base-map tiles on Cloudflare R2, Node 22. Local replica for CPU: Apple M4, Node 24.13, same commit, fed by the production upstreams with the production keys, one synthetic viewer at the front-end’s poll intervals.

Method by quantity.

Quantity	Source	Method
Upstream volumes	production upstreams	three direct polls per feed on 2026-09-30 12:40 to 13:13 UTC; cadence from the replica’s request counters (24 requests in 360 s)
Live vehicles, new fixes	local replica	per-poll log lines and the trace file written during the run
CPU by stage	local replica, profiled	V8 sampling profiler on the main thread, 60 s warm-up excluded, 360 s quiet phase and 360 s viewer phase; functions attributed to stages by file
CPU total	local replica, unprofiled	process CPU sampled every 30 s over 271 s, pollers only
Heap model	production	2,595 health samples at 300 s from 2026-09-14 to 2026-09-30, one uptime segment, first 30 min after start excluded; ordinary least squares
Endpoint bodies	production	five requests per endpoint with brotli and gzip accepted, wire bytes recorded, raw from the decoded body
Per-tab profile	front-end source	poll intervals summed; bytes from the endpoint bodies
Storage	archive	file sizes and line counts over 40 days, medians over the last seven
Fix age	production	report time of every bus in five consecutive responses subtracted from the response time
Learner duration, keys seen and learned	local run	one run on 2026-09-28 over three days of traces (document 1, table 5.1)

Quantity	Source	Method
24-hour profile	local replica, unprofiled, and production	replica run 2026-09-30 21:00 UTC to 2026-10-01 22:00 UTC with process CPU and memory sampled every 30 s and the per-poll log; production bodies fetched every 10 minutes (148 samples); hourly medians in <code>data-24h/diurnal.json</code>

Day and night. Night: 2026-09-29 22:14 to 22:40 UTC. Day: 2026-09-30 12:40 to 13:13 UTC. Same code, machine and method.

Table 5: The two calibration windows.

Quantity	Night	Day
Feed records per poll	8,273	8,454
Live vehicles kept	3,314	6,416
Logged parse ms per poll	44	42
Main-thread busy ms per 15 s	203.7	201.2
Process CPU ms per 15 s, unprofiled	423.7	434.9
Replica heap median, MB	183	193
Vehicle states at end of run	4,466	8,191
Predictions per upstream fetch	4,487	8,084
Bus body, raw / compressed KB	456 / 77	876 / 136
Prediction body, raw / compressed KB	4,788 / 153	7,123 / 217
Fix age at response, median / p95 s	46.5 / 181.5	32 / 155.1

Twenty-four hours. Hourly extremes of the weekday run, UTC.

Table 6: The weekday range of the quantities that drive load; 1 of 5,907 polls failed upstream.

Quantity	Low	High
Live buses per poll	611 at 02:00	7,064 at 15:00
Train predictions per fetch	80 at 01:00	9,331 at 15:00
Bus feed, KB on the wire per poll	1,082	1,144
Process CPU, ms per 15 s	286	596
Heap used, MB	164	415
Bus fix age at response, median s	43.4	53.05

Storage.

Table 7: Storage rates from the archive, 40 days, medians over 2026-09-23 to 2026-09-29.

Dataset	Bytes per day (median of 7)	Records per day	Bytes per record	Per year
Bus traces, as written	763 MB	8,768,876 fixes	86.6	278 GB
Bus traces, archived (gzip)	144 MB	same	16.4	52.6 GB
Rollups	303 KB	2,142 route-directions	142	111 MB
Diversion transitions	1.31 MB	5,411	242	478 MB

Dataset	Bytes per day (median of 7)	Records per day	Bytes per record	Per year
Line status snapshots	4.7 MB	131 snapshots	36 KB	1.7 GB
Leaderboard day archive	2.3 MB	22,930 totals	100	0.8 GB
Learned route snapshot	8.3 MB	about 1,950 files		3.0 GB

Second calibration point. The Dubai deployment runs the same build with no bus feed, no train inference and 213 bike stations: heap 30 MB, resident 117 MB after 15 days of uptime. That is the intercept of Equation 2 without London’s baked geometry and route indexes, and it bounds what the runtime and framework cost on their own.

Appendix B. Stages to code

Table 8: Where each stage lives. The two `shared/` inference files are hand-maintained copies of their front-end counterparts so that the leaderboard ranks by the same positions the map shows.

Stage	Backend (backend/src/)	Front-end (frontend/src/)
1 Ingest	<code>bods-client.ts</code> (SIRI parser, vehicle table), <code>ais-client.ts</code> , <code>gbfs-client.ts</code> , <code>status-recorder.ts</code> , <code>routes/proxy-route.ts</code> (generic proxy), <code>routes/external.ts</code> , <code>tfl-client.ts</code> , <code>rate-budget.ts</code>	
2 State	<code>tfl-cache.ts</code> , tables inside the clients, <code>diversion-detector.ts</code> (states, indexes, gates), <code>leaderboard.ts</code> (buckets)	
3 Detection	<code>diversion-detector.ts</code> , <code>leaderboard.ts</code> + <code>shared/position-inference.ts</code> , <code>nr-sampler.ts</code> + <code>shared/nr-inference.ts</code> , <code>disruption-resolver.ts</code> , <code>bus-stop-closures.ts</code>	
4 Persist	<code>trace-writer.ts</code> , <code>rollup-writer.ts</code> , <code>status-recorder.ts</code> , <code>leaderboard.ts</code> (persist), <code>config.ts</code> (persistPath)	
5 Serve	<code>app.ts</code> (wiring, compression), <code>routes/*.ts</code> , <code>server.ts</code>	
6 Batch	<code>learner-scheduler.ts</code> → <code>scripts/learn-bus-routes.mjs</code> , <code>scripts/fetch-bus-prior.mjs</code> , <code>coverage-writer.ts</code>	
Client		<code>main.ts</code> ; <code>layers/</code> (one file per layer); <code>realtime/</code> (<code>position-inference.ts</code> , <code>interpolator.ts</code> , <code>bus-kalman.ts</code>); <code>util/</code> (<code>lifecycle.ts</code> , <code>render-gate.ts</code>)
Measurement	<code>GET /health</code> (memory and component counters)	